

**DEPARTAMENTUL DE INGINERIE ELECTRICA, ELECTRONICA SI
CALCULATOARE**

Programul de studii:Calculatoare

TESTARE ONLINE

PROIECT: PROIECTAREA APLICATIILOR WEB

Autor: **Pop Lucas**



UNIVERSITATEA TEHNICA

DIN CLUJ-NAPOCA

CENTRUL UNIVERSITAR NORD DIN BAIA MARE

FACULTATEA DE INGINERIE

Cuprins

1	Introducere	3
1.1	Context general	3
1.2	Obiective	3
1.3	Lista MoSCoW	4
1.4	Cazuri de utilizare	4
2	Arhitectura	6
2.1	Front-end React	7
2.2	Back-end Spring Boot	7
2.2.1	Dependency Injection (DI)	8
2.2.2	Spring Security si JWT	9
2.3	Comunicare intre front-end si back-end	9
2.3.1	REST API cu Axios	9
2.3.2	WebSockets pentru Comunicare in Timp Real	10
2.4	Baza de date MySQL	11
2.4.1	Integrare JPA (Java Persistence API)	11
2.5	Beneficii ale acestei arhitecturi	12
3	Implementare Frontend	13
3.1	Tehnologii Frontend	14
3.2	Structura componentelor	14
3.2.1	Componentele folosite	15
3.3	Utilizarea aplicatiei din perspectiva Guest-ului	16
3.4	Utilizarea aplicatiei din perspectiva Administratorului	16
4	Implementare Backend	17
4.1	Tehnologii Backend	17
4.1.1	Spring Boot	17
4.1.2	WebSockets	18
4.1.3	MySQL	19
4.1.4	Maven	19
4.2	Gestionarea cererilor HTTP	19
5	Concluzii	20

1. Introducere

1.1. Context general

Aplicatia web de testare online a fost conceputa pentru a raspunde nevoii actuale de digitalizare a procesului de evaluare a cunostintelor in diverse contexte. In era digitala, evaluarea competentelor si cunostintelor se realizeaza din ce in ce mai mult in mediul online, fie ca este vorba despre educatie, training-uri corporative, certificari profesionale sau evaluari in cadrul proceselor de recrutare.

Aceasta aplicatie ofera o solutie completa atat pentru administratori, care pot crea si gestiona teste in orice domeniu de cunoastere, cat si pentru utilizatori, care pot sustine aceste teste intr-un mediu intuitiv, accesibil si flexibil. Prin digitalizarea procesului de evaluare, se elimina limitarile testelor pe suport fizic si se ofera posibilitatea unei evaluari imediate, obiective si accesibile de oriunde.

1.2. Obiective

Scopul principal al acestei aplicatii web este de a simplifica si automatiza procesul de evaluare a cunostintelor prin intermediul testelor tip grila, indiferent de domeniul sau contextul de aplicare.

Prin dezvoltarea acestei aplicatii se urmareste atingerea urmatoarelor obiective:

- Crearea si editarea facila a testelor online de catre administrator in orice domeniu
- Sustinerea testelor intr-un mod simplu si intuitiv de catre utilizatori
- Generarea automata a rezultatelor si feedback-ului imediat
- Gestionarea eficienta a intrebarilor, testelor si rezultatelor
- Asigurarea unui mediu securizat pentru desfasurarea evaluarilor
- Notificarea administratorului atunci cand se face submit la test
- Oferirea unei interfete moderne si responsive, accesibila de pe orice dispozitiv
- Adaptabilitate la diverse contexte: educational, corporativ, certificari, recrutare

1.3. Lista MoSCoW

În cadrul procesului de dezvoltare a aplicației de testare online, este esențial să stabilim prioritățile și să identificăm cerințele în funcție de importanța lor strategică. Metoda MoSCoW oferă o structură clară pentru clasificarea acestora în patru categorii distincte:

- **Must-haves** (Trebuie să aibă) - funcționalități esențiale pentru aplicație
- **Should-haves** (Ar trebui să aibă) - funcționalități importante, dar nu critice
- **Could-haves** (S-ar putea să aibă) - funcționalități optionale care adaugă valoare
- **Won't-haves** (Nu va avea) - funcționalități excluse din versiunea curentă

Must have	Should have	Could have	Won't have
Autentificare utilizatori	Vizualizarea statisticilor detaliate	Export rezultate în PDF	Integrare cu alte platforme
Operații CRUD pentru teste	Filtrare teste după categorii	Cronometru vizual	Suport pentru întrebări cu răspuns deschis
Operații CRUD pentru întrebări	Setare timp limită per test		Chat live între utilizatori
Sustinerea testelor online			
Afișarea rezultatelor			

Tabela 1: Lista MoSCoW

1.4. Cazuri de utilizare

Această aplicație este destinată atât administratorilor (organizatori de teste, formatori, HR, profesori), cât și utilizatorilor finali (candidați, angajați, cursanți, elevi). În cadrul aplicației am identificat diverse cazuri de utilizare care reflectă scenariile principale pentru utilizatorii noștri.

Aceste cazuri de utilizare ilustrează modul în care atât administratorii, cât și participanții interacționează cu aplicația, evidențind funcționalitățile esențiale și oferind o viziune detaliată asupra experienței utilizatorului.

Cazuri de utilizare pentru Administrator:

- Autentificare în sistem cu credențiale admin

- Creare test nou cu specificarea titlului, descrierii, domeniului si timpului alocat
- Adaugare intrebari la test cu optiuni multiple de raspuns
- Editare sau stergere intrebari existente
- Vizualizare rezultate obtinute de participanti
- Gestionarea listei de utilizatori inregistrati
- Organizarea testelor pe categorii/domenii

Cazuri de utilizare pentru Utilizator/Participant:

- Inregistrare in platforma si autentificare
- Vizualizare lista teste disponibile
- Accesare test si citire instructiuni
- Raspundere la intrebari in timpul alocat
- Trimitere raspunsuri si finalizare test
- Vizualizare rezultat si feedback imediat

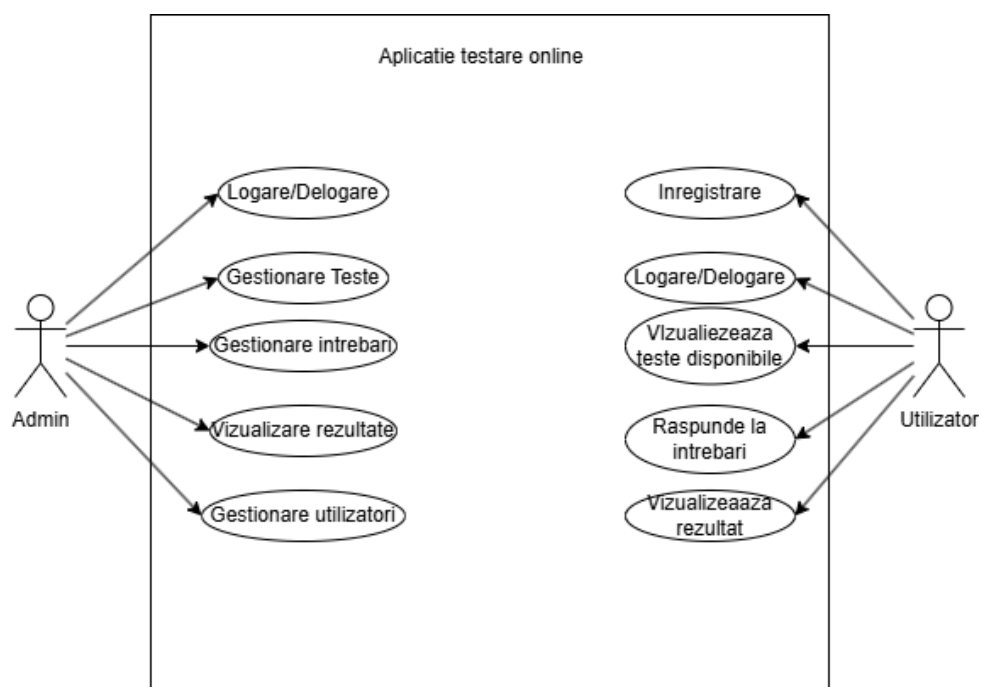


Figura 1: Diagrama cazurilor de utilizare

2. Arhitectura

Arhitectura aplicatiei este conceputa pentru a asigura o performanta ridicata, scalabilitate si o interfata utilizator moderna si intuitiva. Cu un front-end dezvoltat in React si un back-end bazat pe Spring Boot, aceasta arhitectura moderna imbina tehnologii de ultima generatie pentru a oferi o experienta coerenta, robusta si sigura utilizatorilor, indiferent de contextul de utilizare.

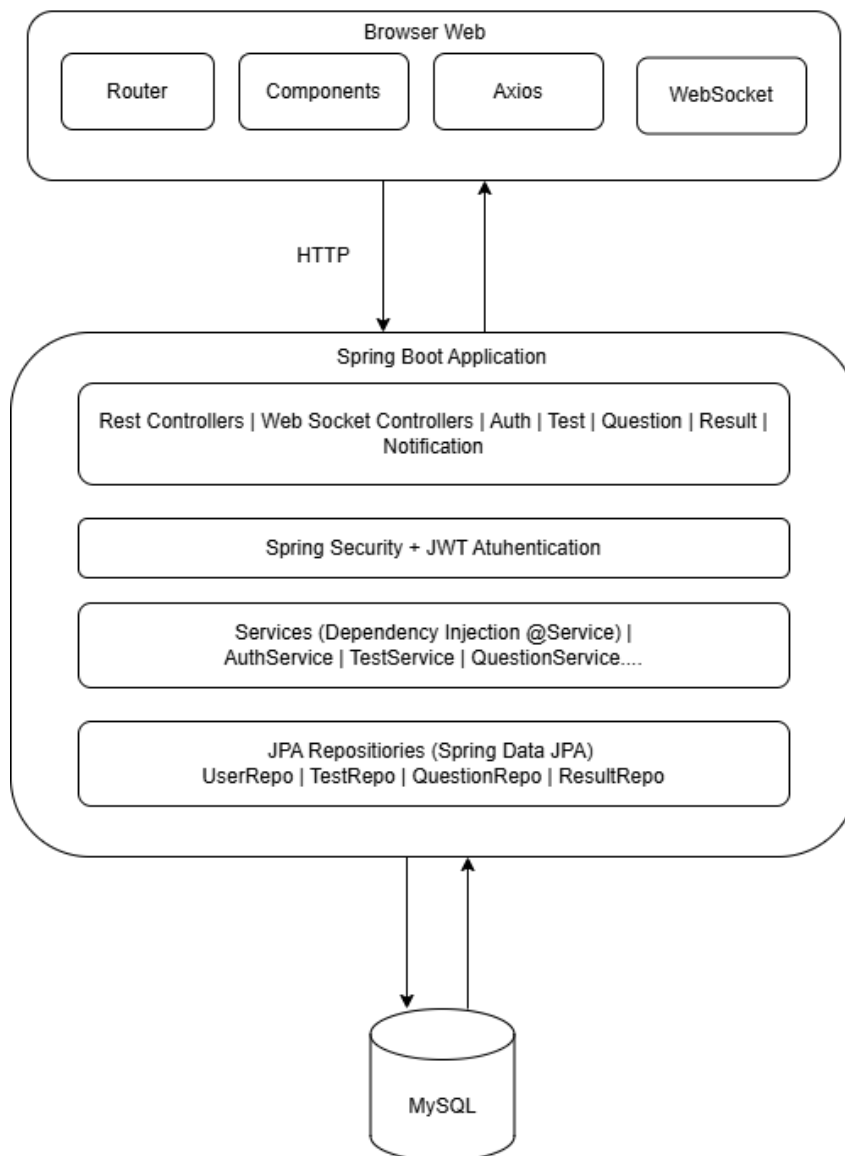


Figura 2: Arhitectura aplicatiei

2.1. Front-end React

React a fost ales pentru dezvoltarea interfeței utilizator datorită modularității, flexibilității și vitezei sale de dezvoltare. Componentele React permit o gestionare eficientă a stării aplicației și facilitarea interacțiunii fluide cu utilizatorul.

Principalele avantaje ale utilizării React includ:

- **Virtual DOM** - Asigură actualizări rapide ale interfeței fără reîncărcarea completă a paginii
- **Componentizare** - Structura bazată pe componente reutilizabile oferă un cod ușor de întreținut și extins
- **Ecosistem bogat** - Acces la numeroase biblioteci și instrumente pentru dezvoltare rapidă
- **Single Page Application (SPA)** - Oferă o experiență fluidă, similară aplicațiilor desktop
- **Suport WebSocket** - Integrare facilă pentru comunicare în timp real cu serverul

Arhitectura front-end-ului include:

- **Router** - Gestionarea navigației între diferitele pagini ale aplicației
- **Components** - Componente reutilizabile pentru interfața utilizator (formulare, tabele, butoane)
- **Axios** - Bibliotecă pentru comunicarea cu back-end-ul prin cereri HTTP
- **WebSocket Client** - Pentru notificări în timp real și actualizări live

2.2. Back-end Spring Boot

Spring Boot reprezintă coloana vertebrală a back-end-ului, furnizând un cadru robust și ușor de utilizat pentru dezvoltarea serviciilor. Utilizând Spring Data JPA, interacțiunea cu baza de date MySQL devine transparentă și eficientă.

Spring Boot oferă următoarele facilități esențiale:

- **Spring REST Controller** - Expunere servicii web prin endpoint-uri RESTful pentru comunicare cu front-end-ul

- **Spring Data JPA** - Simplifica operatiunile CRUD si interactiunea cu baza de date prin mapare obiect-relationala
- **Spring Security** - Asigura autentificarea si autorizarea utilizatorilor prin JWT (JSON Web Tokens)
- **Dependency Injection** - Promoveaza cod modular, testabil si usor de intretinut prin inversiunea controlului
- **Spring WebSocket** - Permite comunicare bidirectionala in timp real intre client si server
- **Configurare automata** - Reduce timpul de setup si permite focus pe logica de business

Arhitectura back-end-ului include:

- **REST Controllers** - Gestioneaza cererile HTTP (GET, POST, PUT, DELETE)
- **WebSocket Controllers** - Gestioneaza conexiunile WebSocket pentru notificari in timp real
- **Services** - Contin logica de business a aplicatiei, injectate prin Dependency Injection
- **Repositories** - Interfete JPA pentru accesarea si manipularea datelor din baza de date
- **Entities** - Modele de date mapate pe tabelele din MySQL prin JPA
- **Security Configuration** - Configurarea JWT pentru autentificare si autorizare

2.2.1. Dependency Injection (DI)

Dependency Injection este un pattern fundamental implementat in Spring Boot care permite injectarea automata a dependentelor intre componente. Prin utilizarea adnotarilor `@Autowired`, `@Service`, `@Repository` si `@Component`, Spring gestioneaza ciclul de viata al obiectelor si dependentele dintre acestea.

Avantajele DI in aplicatia noastra:

- Cod mai usor de testat - dependentele pot fi inlocuite cu mock-uri in teste
- Cuplare slaba intre componente - modificarile in implementare nu afecteaza consumatorii
- Reutilizare crescuta - serviciile pot fi injectate in multiple locuri
- Configurare centralizata - Spring gestioneaza automat instantierea obiectelor

2.2.2. Spring Security si JWT

Securitatea aplicatiei este asigurata prin Spring Security, configurat pentru autentificare bazata pe JWT (JSON Web Tokens). Acest mecanism ofera autentificare stateless, ideala pentru aplicatii REST.

Fluxul de autentificare:

1. Utilizatorul trimite credentiale (username si parola) catre endpoint-ul de login
2. Serverul valideaza credentialele si genereaza un token JWT
3. Token-ul este returnat clientului si stocat in localStorage
4. La fiecare cerere ulterioara, token-ul este trimis in header-ul **Authorization**
5. Serverul valideaza token-ul si permite/respinge accesul la resurse

Beneficii JWT:

- Stateless - serverul nu trebuie sa stocheze sesiuni
- Scalabil - perfect pentru arhitecturi distribuite
- Securizat - token-urile sunt semnate criptografic
- Flexibil - suporta expirare automata si refresh tokens

2.3. Comunicare intre front-end si back-end

Comunicarea intre front-end (React) si back-end (Spring Boot) se realizeaza prin doua mecanisme complementare:

2.3.1. REST API cu Axios

Pentru operatiunile standard CRUD si interogari, comunicarea se face prin cereri HTTP REST, gestionate de biblioteca Axios in React.

Procesul de comunicare include:

- **Cereri HTTP asincrone** - GET (obtinere date), POST (creare resurse), PUT (actualizare), DELETE (stergere)

- **Format JSON** - Schimbul de date se realizeaza in format JSON, usor de procesat
- **CORS** - Configurare pentru permiterea cererilor cross-origin in siguranta
- **Interceptori Axios** - Adauga automat token-ul JWT la fiecare cerere

Abordarea asincrona a Axios contribuie la optimizarea timpului de incarcare si la o experienta de utilizare mai fluida, fara blocarea interfetei in timp ce se asteapta raspunsul de la server.

2.3.2. WebSockets pentru Comunicare in Timp Real

Pentru notificari si actualizari in timp real, aplicatia utilizeaza WebSockets, implementate prin Spring WebSocket pe server si biblioteca SockJS/STOMP pe client.

Cazuri de utilizare WebSocket:

- **Notificari instant** - Administratorul este notificat imediat cand un utilizator finalizeaza un test
- **Actualizari live** - Rezultatele si statisticile se actualizeaza automat fara refresh
- **Monitorizare activitate** - Tracking in timp real al utilizatorilor activi

Implementarea WebSocket:

- **Server** - Spring WebSocket cu STOMP protocol pentru mesaje structurate
- **Client** - SockJS pentru compatibilitate si fallback automat
- **Topics si Queues** - Canale de comunicare pentru broadcast sau mesaje directe

Fluxul comunicarii WebSocket:

1. Clientul React stabileste conexiune WebSocket la pornirea aplicatiei
2. Se subscrie la topic-uri relevante (ex: `/topic/notifications`)
3. Cand un utilizator finalizeaza un test, serverul trimite notificare prin WebSocket
4. Toti clientii subscrisi primesc instant notificarea si actualizeaza UI-ul

2.4. Baza de date MySQL

MySQL a fost ales ca sistem de gestionare a bazelor de date relationale datorita popularitatii sale, performantei si usurintei in utilizare. Interfata Spring Data JPA simplifica interactiunea cu baza de date, oferind o abordare eleganta pentru manipularea datelor intr-un mod eficient si securizat.

Principalele avantaje ale MySQL:

- **Popularitate si suport** - Cea mai utilizata baza de date relationala open-source, cu comunitate vasta
- **Performanta** - Optimizat pentru operatiuni de citire rapide, esentiale pentru aplicatii web
- **Fiabilitate** - Conformitate ACID care garanteaza integritatea datelor prin tranzactii sigure
- **Scalabilitate** - Poate gestiona volume mari de date si multe conexiuni simultane
- **Usurinta in administrare** - Instrumente intuitive pentru gestionarea bazei de date

2.4.1. Integrare JPA (Java Persistence API)

Spring Data JPA ofera o abstractizare de nivel inalt pentru operatiunile cu baza de date, eliminand necesitatea scrierii de cod SQL manual pentru operatiuni comune.

Caracteristici principale JPA:

- **Object-Relational Mapping (ORM)** - Maparea automata intre obiecte Java si tabele MySQL
- **Repository Pattern** - Interfete simple care ofera operatiuni CRUD fara implementare explicita
- **Query Methods** - Generare automata de query-uri bazate pe numele metodelor
- **JPQL** - Limbaj de interogare orientat pe obiecte pentru query-uri complexe
- **Tranzactii** - Gestionare declarativa prin adnotari `@Transactional`

Structura bazei de date include tabele pentru:

- **Users** - Utilizatori (administratori si participanti) cu parole criptate
- **Tests** - Teste cu informatii despre titlu, descriere, timp alocat, categorie
- **Questions** - Intrebări asociate fiecarui test
- **Answers** - Optiuni de raspuns pentru fiecare intrebare (cu marcaj pentru raspunsul corect)
- **TestResults** - Scorurile si raspunsurile utilizatorilor pentru fiecare test
- **UserAnswers** - Raspunsurile individuale ale utilizatorilor la fiecare intrebare

Relatiile intre entitati:

- **User** ↔ **TestResult**: OneToMany (un utilizator poate avea multiple rezultate)
- **Test** ↔ **Question**: OneToMany (un test contine multiple intrebări)
- **Question** ↔ **Answer**: OneToMany (o intrebare are multiple optiuni)
- **TestResult** ↔ **UserAnswer**: OneToMany (un rezultat contine raspunsuri la toate intrebările)

2.5. Beneficii ale acestei arhitecturi

Arhitectura aleasa ofera multiple avantaje care o fac potrivita pentru diverse contexte de utilizare:

Scalabilitate: Arhitectura permite extinderea facila a capacitatilor aplicatiei pe masura ce numarul de utilizatori creste. Separarea front-end-ului de back-end permite scalarea independenta a fiecărei componente. WebSockets gestioneaza eficient mii de conexiuni simultane.

Eficienta in dezvoltare: Utilizarea framework-urilor populare precum React si Spring Boot contribuie la o dezvoltare rapida si usoara. Dependency Injection si JPA reduc cantitatea de cod boilerplate. Comunitatea vasta ofera documentatie bogata si solutii rapide la probleme.

Performanta optimizata: Comunicarea eficienta prin REST si WebSocket, impreuna cu utilizarea MySQL si JPA, asigura timpi de raspuns rapizi. Virtual DOM-ul React si query-urile optimizate JPA contribuie la o experienta fluida.

Flexibilitate si extensibilitate: Modularitatea si structura componentelor permit adaugarea usoara de functionalitati noi. Dependency Injection faciliteaza integrarea de noi servicii fara modificari majore in codul existent.

Mentenabilitate: Separarea clara intre prezentare (React), logica de business (Spring Boot Services) si persistenta (JPA/MySQL) faciliteaza intretinerea si debugging-ul aplicatiei. Codul este organizat, documentat si usor de inteles.

Securitate robusta: Spring Security cu JWT ofera autentificare si autorizare de nivel enterprise. Token-urile JWT sunt semnate criptografic si pot expira automat. MySQL asigura integritatea datelor prin tranzactii ACID.

Comunicare in timp real: WebSockets permit notificari instant si actualizari live, imbunatatind semnificativ experienta utilizatorului. Administratorii sunt notificati imediat cand participantii finalizeaza teste.

Compatibilitate cross-platform: Aplicatia functioneaza pe orice dispozitiv cu browser modern (desktop, tableta, smartphone), fara a necesita instalari suplimentare. React asigura o interfata responsive adaptata la orice rezolutie.

Testabilitate crescuta: Dependency Injection permite injectarea de mock-uri in teste. Separarea responsabilitatilor faciliteaza testarea unitara si de integrare.

Arhitectura noastra reflecta angajamentul fata de o experienta utilizator de calitate superioara si o dezvoltare durabila si eficienta. Prin integrarea tehnologiilor moderne si dovedite (Spring Boot, React, JPA, MySQL, WebSockets, JWT), ne propunem sa construim o aplicatie care sa satisfaca atat nevoile actuale, cat si cerintele viitoare ale utilizatorilor nostri, oferind flexibilitate pentru evolutii si imbunatatiri continue in diverse contexte de utilizare: educational, corporativ, certificari profesionale sau procesele de recrutare.

3. Implementare Frontend

Dezvoltarea interfeței utilizator a fost realizată având ca obiectiv principal crearea unei experiențe fluide, intuitive și responsive. Implementarea frontend-ului se concentrează pe modularitate și reutilizabilitate, respectând principiile moderne de design și arhitectură software.

3.1. Tehnologii Frontend

Pentru realizarea interfeței grafice s-au utilizat tehnologii cunoscute din ecosistemul JavaScript

- **React.js:** Biblioteca principală pentru construirea interfeței. Alegerea React a permis dezvoltarea unei aplicații de tip SPA (Single Page Application), asigurând o navigare rapidă fără reîncărcarea paginii. Utilizarea hook-urilor (useState, useEffect, useContext) a simplificat gestionarea stării componentelor și ciclului de viață al acestora.
- **React Router Dom:** Implementat pentru gestionarea rutării dinamice în aplicație. Acesta permite definirea rutelor protejate (Protected Routes), asigurând că utilizatorii neautentificați nu pot accesa resurse sensibile, cum ar fi panourile de administrare sau testele propriu-zise.
- **Axios:** Utilizat pentru efectuarea cererilor HTTP asincrone către backend-ul Spring Boot. S-a configurat un interceptor global care atașează automat token-ul JWT în header-ul de autorizare pentru fiecare cerere, securizând astfel comunicarea.
- **CSS:** Pentru stilizare, s-a optat pentru CSS modern, implementând un design de tip "Glassmorphism".
- **Context API:** Folosit pentru gestionarea stării globale a aplicației, în special pentru stocarea informațiilor despre utilizatorul autentificat și sesiunea curentă, eliminând necesitatea propagării datelor prin "props drilling".

3.2. Structura componentelor

Arhitectura codului sursă este organizată pe module funcționale, fiecare director având o responsabilitate clară. Această structură facilitează mentenanța și scalabilitatea proiectului. Structura directoarelor principale este următoarea:

- **src/components/admin:** Conține componente specifice rolului de administrator (gestionare teste, statistici, utilizatori).
- **src/components/auth:** Gestionează procesele de autentificare (Login, Signup)
- **src/components/quiz:** Componente legate de logica testelor (creare, editare, listare).

- **src/components/user:** Interfețe pentru utilizatorul final (participare la test, vizualizare rezultate).
- **src/contexts:** Provideri pentru starea globală application-wide.

3.2.1. Componentele folosite

Aplicația este construită din componente React interconectate. Principalele componente implementate sunt:

- **AuthContext:**
 - **Rol:** Componentă de tip Provider care învelește întreaga aplicație.
 - **Funcționalitate:** Verifică la inițializare existența token-ului JWT în localStorage, decodifică rolul utilizatorului și expune metodele de login și logout către restul componentelor.
- **ProtectedRouter:**
 - **Rol:** Gardian de securitate la nivel de rută.
 - **Funcționalitate:** Verifică dacă utilizatorul este autentificat și dacă are rolul necesar pentru a accesa ruta solicitată. În caz contrar, redirecționează utilizatorul către pagina de Login
- **Dashboard:**
 - **Rol:** Pagina principală a aplicației după autentificare.
 - **Funcționalitate:** Redă o interfață adaptivă. Pentru Administratori afișează panouri de control (Statistici, Gestionare Utilizatori), iar pentru Utilizatori afișează opțiunile de participare la teste și istoricul rezultatelor.
- **Quiz & TakeQuiz:**
 - **Rol:** Motorul de desfășurare a testelor.
 - **Funcționalitate:** Quiz.jsx permite administratorului să definească întrebări și variante de răspuns (inclusiv suport pentru răspunsuri multiple). TakeQuiz.jsx randează întrebările pentru student, gestionează cronometrul și trimite răspunsurile către server pentru evaluare.
- **Statistics & MyResults:**

- **Rol:** Vizualizarea performanței.
- **Funcționalitate:** Administratorul are acces la o vizualizare agregată a performanței studenților, cu posibilitatea de a analiza răspunsurile individuale. Utilizatorul își poate vedea propriul istoric în MyResults, cu detalii despre scor, întrebări greșite și punctaj.

3.3. Utilizarea aplicației din perspectiva Guest-ului

Utilizatorul neautentificat (Guest) are un acces limitat strict la funcționalitățile publice ale aplicației, din motive de securitate.

Scenariul de utilizare pentru Guest include:

- **Accesare Landing Page:** La accesarea URL-ului aplicației, guest-ul este întâmpinat de o interfață modernă care prezintă platforma.
- **Autentificare/Înregistrare:**
 - Guest-ul poate naviga către pagina Login, unde trebuie să introducă credențialele.
 - Dacă nu are cont, poate accesa secțiunea Signup. Aici, formularul validează datele în timp real (format email, complexitate parolă) și trimite o cerere POST către backend pentru crearea contului cu rolul default de USER.
- **Restricții:** Orice încercare de a accesa direct o rută privată (ex: /dashboard sau /quiz/take) va declanșa mecanismul ProtectedRoute care va redirecționa automat utilizatorul către pagina de Login.

3.4. Utilizarea aplicației din perspectiva Administratorului

Rolul de Administrator conferă control total asupra platformei și conținutului educațional. Interfața administratorului este optimizată pentru productivitate și monitorizare.

Fluxurile principale de lucru pentru Administrator sunt:

- **Gestionarea Testelor (Quiz Management):**
 - Administratorul poate crea teste noi, definind titlul, descrierea, categoria și limita de timp.

- **Adăugarea Întrebărilor:** Interfața permite adăugarea dinamică a întrebărilor. Pentru fiecare întrebare, admin-ul definește tipul (Single/Multiple Choice), textul întrebării și variantele de răspuns, marcându-le pe cele corecte.
- Editare/Ștergere: Testele existente pot fi modificate sau eliminate din sistem.

- **Monitorizare si Statistici:**

- Secțiunea **Statistics** oferă o privire de ansamblu asupra activității. Sunt afișate carduri cu numărul total de utilizatori, teste create și media notelor.
- **Student Results:** O funcționalitate avansată permite administratorului să vadă lista studenților. Prin expandarea unui student, se afișează istoricul testelor date de acesta. Se pot vizualiza detaliat răspunsurile date de student versus răspunsurile corecte ("View Details"), permițând o evaluare calitativă a procesului de învățare.

- **Gestionarea Utilizatorilor (User Management):**

- Prin componenta **UserList**, administratorul vizualizează toți utilizatorii înregistrați.
- **Editare Utilizator:** Administratorul poate invoca un modal (fereastră pop-up) pentru a modifica datele unui utilizator (Nume, Email) sau pentru a-i schimba rolul (promovare la ADMIN sau retrogradare la USER). De asemenea, poate reseta parolele utilizatorilor dacă este necesar.

4. Implementare Backend

Backend-ul aplicației reprezintă nucleul funcțional, responsabil de logica de business, securitate și persistența datelor. Arhitectura server-side este construită pe o stivă tehnologică robustă, bazată pe ecosistemul Java și Spring, asigurând scalabilitate și mentenabilitate.

4.1. Tehnologii Backend

Selecția tehnologiilor a fost făcută pentru a asigura o integrare perfectă între componente și o performanță optimă.

4.1.1. Spring Boot

Spring Boot a fost ales ca framework principal datorită capacității sale de "Convention over Configuration", care elimină necesitatea configurărilor boilerplate complexe. Acesta

orchestrează toate modulele backend-ului.

Spring Data JPA Componenta de persistență utilizează Spring Data JPA pentru a abstractiza interacțiunea cu baza de date. Prin utilizarea interfețelor de tip **Repository** (ex: **QuizRepository**, **UserRepository**), s-a eliminat codul SQL manual. Framework-ul generează automat query-uri optimizate pe baza numelor metodelor (ex: **findByUsername**), iar pentru interogări complexe s-a folosit JPQL, asigurând independența de dialectul specific bazei de date.

Dependency Injection (DI) Mecanismul de Dependency Injection, prin containerul Inversion of Control (IoC) al Spring, gestionează ciclul de viață al bean-urilor. Serviciile (**QuizService**, **UserService**) sunt injectate automat în Controllere, decuplând logica de business de interfața REST. Această abordare a facilitat testarea unitară și modularizarea codului.

Spring Security Securitatea este implementată la nivel de filtru HTTP. S-a configurat un lanț de filtre (**SecurityFilterChain**) care interceptează fiecare cerere.

- **AuthenticationManager:** Validează credențialele utilizatorilor.
- **JWT Filter:** Un filtru personalizat (**JwtAuthenticationFilter**) extrage token-ul din header, îl validează criptografic și setează contextul de securitate (**SecurityContextHolder**), permițând accesul doar utilizatorilor legitimi.

4.1.2. WebSockets

Pentru a oferi o interactivitate superioară aplicațiilor REST tradiționale, s-a integrat suportul pentru WebSockets.

Implementarea comunicatiei WebSocket Configurarea s-a realizat prin clasa **WebSocketConfig**, care activează brokerul de mesaje simplu în memorie.

- Endpoint-ul de conectare **/ws** permite clienților React să inițieze o conexiune persistentă.
- Brokerul **/topic** este utilizat pentru mesaje de tip publish-subscribe.

- S-a utilizat protocolul STOMP (Simple Text Oriented Messaging Protocol) peste WebSocket, oferind o structură standardizată mesajelor, facilitând serializarea/deserializarea automată a obiectelor Java în JSON și invers.

4.1.3. MySQL

Ca sistem de gestiune a bazelor de date s-a optat pentru MySQL, datorită conformității ACID și maturității sale. Modelul de date relațional a fost proiectat pentru a asigura integritatea referențială (chei străine între Tabelele **Users**, **Tests**, **Questions**) și eficiența interogărilor. Utilizarea strategiilor de generare a ID-urilor (**GenerationType.IDENTITY**) asigură unicitatea cheilor primare.

4.1.4. Maven

Managementul dependențelor și automatizarea procesului de build sunt asigurate de Apache Maven. Fișierul **pom.xml** centralizează toate librăriile necesare (Spring Boot Starter Web, Security, Data JPA, MySQL Connector, Lombok, JWT), garantând consistența versiunilor și reproductibilitatea build-ului pe orice mediu. Plugin-ul **spring-boot-maven-plugin** permite împachetarea aplicației într-un fișier JAR executabil independent.

4.2. Gestionarea cererilor HTTP

Interacțiunea cu frontend-ul se realizează exclusiv prin API-uri REST, implementate în clase anotate cu **@RestController**.

Fiecare controller gestionează o resursă specifică:

- **AuthController:** Gestionează rutele publice **/api/auth/login** și **/api/auth/signup**.
- **QuizController:** Manipulează resursele educaționale, oferind endpoint-uri pentru CRUD operațiuni pe teste și întrebări.
- **AdminController:** Expune funcționalități sensibile, protejate prin adnotarea **@PreAuthorize("hasRole('ADMIN')")**, asigurând că doar administratorii pot accesa datele statistice sau modifica alți utilizatori.

5. Concluzii

Proiectul dezvoltat reprezintă o platformă completă și modernă de testare online, care demonstrează integrarea cu succes a unor tehnologii de actualitate în dezvoltarea software. Prin combinarea puterii și robusteții backend-ului Spring Boot cu interactivitatea și dinamismul frontend-ului React, s-a obținut o soluție software performantă, sigură și scalabilă.

Obiectivele inițiale ale proiectului au fost atinse în totalitate:

- **Arhitectură Modernă:** Implementarea unei arhitecturi distribuite (Client-Server) bazată pe REST API și microservicii logice.
- **Experiență Unitară:** Utilizarea Glassmorphism și a design-ului responsive asigură o utilizare plăcută pe orice dispozitiv
- **Securitate Avansată:** Implementarea autentificării JWT și a rutelor protejate garantează confidențialitatea datelor și integritatea procesului de examinare.
- **Interactivitate:** Funcționalitățile în timp real și feedback-ul instant la completarea testelor îmbunătățesc procesul educațional.

Aplicația oferă o bază solidă pentru dezvoltări ulterioare, având potențialul de a fi extinsă cu funcționalități precum generarea automată a certificatelor, integrarea cu platforme de e-learning externe sau implementarea unor algoritmi de inteligență artificială pentru analiza performanței studenților.

În concluzie, acest proiect nu este doar o demonstrație tehnică a competențelor de programare Full Stack, ci și un produs software viabil, capabil să răspundă nevoilor reale de digitalizare a proceselor de evaluare și testare.

[LINK GITHUB](#)